

CS271 Winter 2022

Final Project

Assigned: February 9, 2022
Due on Demo Day: Monday March 14

PLEASE NOTE:

- This programming assignment can be done in teams of TWO.

1 Introduction

In this final project, you will develop a distributed highly available chat system replicated on a set of clients. The clients are responsible for maintaining a log of encrypted messages using **Raft**. Clients will be able to join and leave group chats and only clients within a group will have access to the messages of that group.

1.1 Application Components

Since *Raft* is a *leader-based* consensus protocol, the project can be decomposed into three parts:

- Leader Election
- Normal Operations
- Fault Tolerance

In the first part, you will implement the leader election of the Raft protocol. In the second part, you will implement the normal operations of the Raft protocol to maintain a log that will be used to store messages and group updates. In the last part, you will deal with the node failures and network partition failures of Raft protocol while at the same time, supporting normal operations.

You should prioritize the assignment parts in this order:

1. Implement Raft
2. Support persistent disk storage of Raft's state (e.g. in a file on disk)
3. Support site crash and recovery from disk

4. Support group chat with no privacy
5. Support group chat with privacy using public-private key cryptography
6. Support networking partitioning and recovery from a partition

1.2 Leader Election

There will be five clients starting as five followers at the beginning.

Following the Raft leader election protocol, there will be exactly one leader, and four followers after the leader election stage.

1.3 Normal Operations

In order to execute normal operations, you must correctly implement the Raft protocol first.

1.3.1 Clients

Clients are interested in joining groups and multi-casting chat messages to members of specific groups. Each of the five clients will maintain and persist the following state on disk:

- The client's personal public and private keys
- The public keys of all other clients (you can either choose to publish this to the log at startup or keep it separately)
- A copy of the replicated log and other state needed for Raft (e.g. `currentTerm` & `votedFor`)

At initialization, the clients will all be directly connected to each other in a mesh network. Note that due to failures this initial configuration may change (more below).

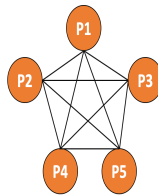


Figure 1: All processes directly connected to each other at initialization

1.3.2 Creating a group and adding/kicking members

- **Creating a group.** Any client can decide at any time that it wants to create a new group. This client will first create a globally unique `group_id` for that group (e.g. using its pid + a persistent counter). It will then generate a public-private key pair for that group. It will choose to add a number of other clients to the group as group members. It will encrypt the private key of the group with the public keys of each of the members of the group. Finally, it will add the `group_id`, the associated client ids, the public key for the group, and all of the encrypted versions of the private key to the log.
- **Adding a member to a group.** A client belonging to a group can add another client with `client_id` to that group. To do this, it will encrypt the group's private key with their public key and publish it to the log, along with information that `client_id` joined `group_id`.
- **Kicking a member from a group.** A client P_2 can kick to another client P_1 causing them to leave the group. P_2 will be responsible for creating a new group with the same `group_id`, but with P_1 excluded. In order to revoke read permission for future messages from P_1 , P_2 will also have to generate a new public-private key pair for the group and encrypt it with the remaining members' public keys. P_2 will finally publish all of this information as a "leave event" to the log. We will assume that P_2 is a member of the group from which it is kicking P_1 , in theory, the other members of the group could check if this is the case before accepting P_2 's actions.

1.3.3 Reading and Writing messages to a group

- **Reading messages.** At any point in time, a client should be able to read (print to the console) all of the messages that it has access to for a given group with `group_id`. The client will need to decrypt the messages using the private key for the group. You will need to be careful for the case when a client leaves a group, you will need to use multiple private keys to decrypt all of the messages you have access to in the group. The client does not need to store private keys for groups separately, you should be able to do this by traversing the log. Note that the client must also first decrypt the group private key(s) with its private key.
- **Writing messages.** At any point in time, a client should be able to write a message to a group by encrypting its message with the latest public key for the group and then appending the encrypted message, `group_id`, and its `client_id` to the log. While a client that is not part of a group can still encrypt messages with the group's public key and add them to the log, the existing group members can simply ignore these messages as they read the group chat since they know which members are in the group at any point in time.

1.3.4 Replicated Log

You will create a replicated log that will be maintained on all five clients via the Raft protocol. It will contain the following types of entries:

- **Create entry.** Containing the `group_id`, group member `client_ids`, group public key, encrypted versions of the group private key with the group members' public keys
- **Add entry.** Containing the `group_id`, added client's `client_id`, encrypted versions of the group's private key with the joining client's public key
- **Kick entry.** Containing the `group_id`, kicked member's `client_id`, new group public key, encrypted versions of the new group private key with the remaining clients' public keys
- **Message entry.** Containing the `group_id`, sender's `client_id`, and the encrypted message using the group's latest public key

You do not necessarily have to follow the structure above and you may split up the data according to what best suites your application. Note that in addition to application data, each entry will also need to contain data required by Raft (e.g. the `term` and `index`).

1.4 Fault Tolerance

The clients should be able to perform normal operations even if the following failures happen:

1.4.1 Node Failure

- **Follower failure.** This is where a non-leader node fails. After a follower fails, the program should still be able to perform normal operations that don't require that follower.
- **Leader failure.** This is where a leader fails. After leader failure, Raft will begin a new term with leader election after which it should be able to perform normal operations.

1.4.2 Network Partition

Due to network failures, the clients can be decomposed into multiple partitions. In one scenario, one of the partitions has a majority of clients (i.e. ≥ 3 clients). If the leader is amongst them then this scenario behaves like a follower failure and if it is not amongst them then this scenario behaves like a leader failure and a new leader must be elected to continue normal operation. In another scenario, none of the partitions have a majority. In this case, no partition can elect a leader and thus normal operation stops until these partitions join to form a majority.

1.4.3 Node Recovery

After a node recovers from a failure or connects to the majority partition after a network partition, it needs to update its log and resume normal operation. You can decide how to handle this recovery but we expect you to store the log as well as any private and public keys on disk. This will allow you to recover from a reasonable state in case all processes fail. The client that just recovered needs to be able to read any messages it may have missed. This should be the case even if the client was added to a group while it was in a crashed state.

2 Implementation Details

- Please add a delay 3 second delay before each message to simulate network delays.
- You can use an existing libraries in your language of choice to do public-private key cryptography

3 User Interface

The user must be able to input the following commands:

1. **createGroup** <group_id>... : A client receiving this command should create a group using the list of **group_ids** specified in the command.
2. **add** <group_id> <client_id> : A client receiving this command should add the client with **client_id** to the group with **group_id**.
3. **kick** <group_id> <client_id> : A client receiving this command should kick the client with **client_id** from the group with **group_id**.
4. **writeMessage** <group_id> <message> : Writes encrypts a message to a group using that group's latest public key to encrypt the message.
5. **printGroup** <group_id>: This command should print the **client_ids** for all of the current group members as well as a complete transcript for all of the messages it was able to read throughout its membership in the group. You should display the sender along with the corresponding message text as you would see in a regular chat application.
6. **failLink** <src> <dest>: This command must emulate a communication link failure between the **src** process and **dest** process. For example: if we want the communication link between P1 and P2 to fail, we will use this interface as a user input on process P1. And your program should then treat the link between the two nodes P1 and P2 as failed and hence should not send any message between the two nodes. For simplicity, links are considered bidirectional and breaking a link allows neither the **src** nor the **dest** to communicate with each other.

7. **fixLink <src> <dest>**: This is a counterpart of failLink input. This input fixes a broken communication link between any two processes, upon which the two nodes will be able to communicate with each other again. This input will be provided on the **src** process.

A suggestion to simulate such a behaviour could be to use a dictionary data structure that has the network links in the system and has an active or non-active boolean. You can use such a map to check if the network link is active before sending out any message. This is one suggestion; you can use any other technique as long as the behaviour of network failure and fixing of it is emulated.

8. **failProcess**: This input kills the process to which the input is provided. You will be asked to restart the process after it has failed. The process should resume from where the failure had happened and the way to do this would be to store the state of a process on disk and reading from it when the process starts back.

4 Deadlines and Deployment

This project will be due Monday March 14. We will also make it optional to demo on the last day of instruction **Friday March 11** if you are ready by then. We will have a short demo for each project For this project's demo via Zoom (sign-up sheet for demo time slots and zoom link will be posted on piazza). You can deploy your code on several machines. However it is also acceptable if you just use several processes in the same machine to simulate the distributed environment.